Diseño y Análisis de Algoritmos

Ayudantía 4: Hashing y Búsqueda de Patrones (KMP)

Ayud. Ferran Rojas Andreu

1. Cálculo de los valores hash

k	$h(k) = k$ mód 7	posición base
22	22 mód 7 = 1	1
12	12 mód 7 = 5	5
14	14 mód 7 = 0	0
5	5 mód 7 = 5	5
20	20 mód 7 = 6	6
7	7 mód 7 = 0	0

2. (a) Secuencias de sondeo e inserción

Se inserta en el orden dado. El sondeo lineal examina las posiciones $(h(k) + j)$ mód 7 para $j = 0, 1, 2, \ldots$ hasta encontrar una casilla vacía.

22: base 1, vacía ⇒ se inserta en 1. *Sonda:* 1.

12: base 5, vacía ⇒ se inserta en 5. *Sonda:* 5.

14: base 0, vacía ⇒ se inserta en 0. *Sonda:* 0.

5: base 5, ocupada por 12 ⇒ sondea 6, vacía ⇒ se inserta en 6. *Sonda:* $5, 6$.

20: base 6, ocupada por 5 ⇒ sondea 0 (ocupada por 14), 1 (ocupada por 22), 2, vacía ⇒ se inserta en 2. *Sonda:* $6, 0, 1, 2$.

7: base 0, ocupada por 14 ⇒ sondea 1 (ocupada por 22), 2 (ocupada por 20), 3, vacía ⇒ se inserta en 3. *Sonda:* $0, 1, 2, 3$.

3. (b) Estado final de la tabla

Índice	0	1	2	3	4	5	6
Clave	14	22	20	7	_	12	5

Solo la casilla 4 queda vacía. La tabla final es:

slot 0:14, slot 1:22, slot 2:20, slot 3:7, slot 4:∅, slot 5:12, slot 6:5.

4. (c) Factor de carga y comentario sobre sondeo lineal

Se insertaron $n = 6$ claves en $m = 7$ casillas:

$$\alpha = \frac{n}{m} = \frac{6}{7} \approx 0{,}857.$$

Aunque $\alpha < 1$, el sondeo lineal sufre de *agrupamiento primario*: una vez que se forma un bloque contiguo de casillas ocupadas (aquí el bloque $0, 1, 2, 3$), las nuevas claves que caen en cualquier posición del bloque terminan sondeando secuencialmente todo el bloque, alargando el número esperado de comparaciones. Con α cercano a 1, el costo esperado de una búsqueda exitosa en sondeo lineal crece hasta $\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$, que diverge a medida que la tabla se llena.

5. (a) Cálculo de la función prefijo (LPS) de $P =$ `ababaca`

Sea $P[1..7] = \langle a, b, a, b, a, c, a \rangle$. Inicialmente $k = 0$ y $\pi[1] = 0$.

q	comparaciones $P[k+1]$ vs $P[q]$	k final	$\pi[q]$
2	$P[1]{=}a \neq P[2]{=}b$	0	0
3	$P[1]{=}a = P[3]{=}a \Rightarrow k \leftarrow 1$	1	1
4	$P[2]{=}b = P[4]{=}b \Rightarrow k \leftarrow 2$	2	2
5	$P[3]{=}a = P[5]{=}a \Rightarrow k \leftarrow 3$	3	3
6	$P[4]{=}b \neq P[6]{=}c$; $k \leftarrow \pi[3]{=}1$; $P[2]{=}b \neq c$; $k \leftarrow \pi[1]{=}0$; $P[1]{=}a \neq c$	0	0
7	$P[1]{=}a = P[7]{=}a \Rightarrow k \leftarrow 1$	1	1

Por lo tanto:

$$\boxed{\pi \;=\; \langle\, 0,\; 0,\; 1,\; 2,\; 3,\; 0,\; 1 \,\rangle}.$$

Lectura: por ejemplo, $\pi[5] = 3$ indica que el mayor prefijo propio de `ababa` que también es sufijo es `aba` (largo 3).

6. (b) Ejecución de KMP-Matcher sobre $T =$ `abababacaba`

$n = 11$, $m = 7$, $\pi = \langle 0, 0, 1, 2, 3, 0, 1 \rangle$. Se recorre el texto de izquierda a derecha manteniendo q = largo del prefijo de P que coincide con el sufijo de $T[1..i]$:

i	$T[i]$	acción (mismatch $\Rightarrow q = \pi[q]$)	q final
1	a	$P[1]{=}a = T[1] \Rightarrow q \leftarrow 1$	1
2	b	$P[2]{=}b = T[2] \Rightarrow q \leftarrow 2$	2
3	a	$P[3]{=}a = T[3] \Rightarrow q \leftarrow 3$	3
4	b	$P[4]{=}b = T[4] \Rightarrow q \leftarrow 4$	4
5	a	$P[5]{=}a = T[5] \Rightarrow q \leftarrow 5$	5
6	b	$P[6]{=}c \neq b$; $q \leftarrow \pi[5]{=}3$; $P[4]{=}b = T[6] \Rightarrow q \leftarrow 4$	4
7	a	$P[5]{=}a = T[7] \Rightarrow q \leftarrow 5$	5
8	c	$P[6]{=}c = T[8] \Rightarrow q \leftarrow 6$	6
9	a	$P[7]{=}a = T[9] \Rightarrow q \leftarrow 7 = m$	**match**, $q \leftarrow \pi[7]{=}1$
10	b	$P[2]{=}b = T[10] \Rightarrow q \leftarrow 2$	2
11	a	$P[3]{=}a = T[11] \Rightarrow q \leftarrow 3$	3

En $i = 9$ se alcanza $q = m = 7$:

$$\text{shift} = i - m = 9 - 7 = 2.$$

Efectivamente, $T[3..9] =$ `ababaca` $= P$. No hay más ocurrencias.

$$\boxed{\text{Única ocurrencia: shift 2, es decir } T[3..9]}.$$

7. (c) Complejidad de CLRS

Compute-Prefix-Function. El contador k crece a lo más una unidad por iteración del `for` (hasta m), y el `while` solo lo decrece vía $k = \pi[k]$. En total el `while` se ejecuta a lo más m veces (amortizado), luego el costo es

$$\Theta(m).$$

KMP-Matcher. Por el mismo argumento aplicado a q (crece a lo más una vez por posición de T y decae solo vía π), el bucle `for` de n pasos más los `while` acumulan

$$\Theta(n+m).$$

Conclusión. La función prefijo π (LPS) se calcula en $\Theta(m)$ y el matching completo corre en $\boxed{\Theta(n+m)}$, que es lineal en la longitud total del texto y el patrón.

8. (a) Depuración del rastreo de Compute-Prefix-Function

La tabla del estudiante es (valores incorrectos marcados):

q	k al inicio	evolución de k	$\pi[q]$ reportado
2	0	$P[1]{=}a \neq P[2]{=}b$	0
3	0	$P[1]{=}a = P[3]{=}a \Rightarrow k = 1$	1
4	1	$P[2]{=}b = P[4]{=}b \Rightarrow k = 2$	$\boxed{1}$ (debe ser 2)
5	1	$P[2]{=}b \neq P[5]{=}a;\ k = \pi[1] = 0;\ P[1]{=}a = P[5]{=}a \Rightarrow k = 1$	$\boxed{1}$ (debe ser 3)
6	1	$P[2]{=}b \neq P[6]{=}c;\ k = \pi[1] = 0;\ P[1]{=}a \neq P[6]{=}c$	0
7	0	$P[1]{=}a = P[7]{=}a \Rightarrow k = 1$	1

Los dos valores incorrectos son $\pi[4]$ y $\pi[5]$.

Causa raíz. En $q = 4$, el paso correcto es $k \leftarrow k + 1$ (línea `if` del pseudocódigo): como $P[k{+}1] = P[2]{=}b$ coincide con $P[4]{=}b$, debe quedar $k = 2$ y por tanto $\pi[4] = 2$. El estudiante anotó $k = 1$, propagando ese valor erróneo a $q = 5$: al entrar con $k = 1$, el algoritmo nunca compara $P[3]$ contra $P[5]$, de modo que $\pi[5]$ se reporta 1 en vez de 3. Las filas $q = 6$ y $q = 7$ *se recuperan por coincidencia* y quedan correctas.

Función prefijo correcta:

$$\boxed{\pi = \langle\, 0,\ 0,\ 1,\ 2,\ 3,\ 0,\ 1 \,\rangle}.$$

9. (b) Depuración del rastreo de KMP-Matcher

La tabla del estudiante es (valores incorrectos marcados):

i	$T[i]$	acción	q final reportado
1	a	$P[1]{=}a = T[1] \Rightarrow q = 1$	1
2	b	$P[2]{=}b = T[2] \Rightarrow q = 2$	2
3	a	$P[3]{=}a = T[3] \Rightarrow q = 3$	3
4	b	$P[4]{=}b = T[4] \Rightarrow q = 4$	4
5	a	$P[5]{=}a = T[5] \Rightarrow q = 5$	5
6	b	$P[6]{=}c \neq T[6];\ q = \pi[5] = 3;\ P[4]{=}b = T[6] \Rightarrow q = 4$	$\boxed{3}$ (debe ser 4)
7	a	$P[5]{=}a = T[7] \Rightarrow q = 5$	5
8	c	$P[6]{=}c = T[8] \Rightarrow q = 6$	6
9	a	$P[7]{=}a = T[9] \Rightarrow q = 7 = m$; imprime shift 2; $q = \pi[7] = 1$	$\boxed{7}$ (debe ser 1)
10	b	$P[2]{=}b = T[10] \Rightarrow q = 2$	2
11	a	$P[3]{=}a = T[11] \Rightarrow q = 3$	3

Los dos valores incorrectos están en $i = 6$ y $i = 9$.

- $i = 6$**:** el estudiante aplicó el retroceso $q = \pi[5] = 3$ pero *omitió el* `if` *posterior* (línea `If` de KMP-Matcher): como $P[q{+}1] = P[4]{=}b = T[6]{=}b$, debe quedar $q = 4$, no 3.
- $i = 9$**:** el estudiante detectó el match pero *omitió el reinicio* $q \leftarrow \pi[q]$ (línea después del `print`): tras reportar la ocurrencia en el shift 2, debe ser $q = \pi[7] = 1$, no $q = 7$.

Secuencia correcta de q:

$$\boxed{q = \langle\, 1,\ 2,\ 3,\ 4,\ 5,\ 4,\ 5,\ 6,\ 1,\ 2,\ 3 \,\rangle}.$$

La única ocurrencia se encuentra en $i = 9$, con

$$\boxed{\text{shift} = i - m = 9 - 7 = 2}.$$

10. (c) Por qué es peligroso no reiniciar q en $i = 9$

Si al imprimir el match se dejara $q = 7$ sin reiniciar, en $i = 10$ el algoritmo intentaría comparar $P[q{+}1] = P[8]$, que *no existe* (el patrón tiene largo $m = 7$): se accede fuera de rango del arreglo. Aun si la implementación lo evitara, $q = 7$ le haría creer que un prefijo completo del patrón sigue coincidiendo y reportaría desplazamientos espurios (falsos positivos) o fallaría el invariante $q \leq m$ que garantiza el costo lineal. Por eso, tras cada match, KMP restablece $q = \pi[q]$.

11. (a) Función prefijo de $S = P\#T$

Con $P =$ `ababaca` ($m = 7$) y $T =$ `abababacaba` ($n = 11$):

$$S = \texttt{a b a b a c a \# a b a b a b a c a b a}, \qquad |S| = m + 1 + n = 19.$$

Aplicando COMPUTE-PREFIX-FUNCTION al string S completo (recordar que en la tabla $S[k+1]$ cumple el rol de $P[k+1]$):

q	$S[q]$	evolución de k	$\pi[q]$
1	a	(caso base)	0
2	b	$S[1]{=}a \neq b$	0
3	a	$S[1]{=}a = a \Rightarrow k = 1$	1
4	b	$S[2]{=}b = b \Rightarrow k = 2$	2
5	a	$S[3]{=}a = a \Rightarrow k = 3$	3
6	c	$S[4]{=}b \neq c;\ k = \pi[3]{=}1;\ S[2]{=}b \neq c;\ k = \pi[1]{=}0;\ S[1]{=}a \neq c$	0
7	a	$S[1]{=}a = a \Rightarrow k = 1$	1
8	$\#$	$S[2]{=}b \neq \#;\ k = \pi[1]{=}0;\ S[1]{=}a \neq \#$	0
9	a	$S[1]{=}a = a \Rightarrow k = 1$	1
10	b	$S[2]{=}b = b \Rightarrow k = 2$	2
11	a	$S[3]{=}a = a \Rightarrow k = 3$	3
12	b	$S[4]{=}b = b \Rightarrow k = 4$	4
13	a	$S[5]{=}a = a \Rightarrow k = 5$	5
14	b	$S[6]{=}c \neq b;\ k = \pi[5]{=}3;\ S[4]{=}b = b \Rightarrow k = 4$	4
15	a	$S[5]{=}a = a \Rightarrow k = 5$	5
16	c	$S[6]{=}c = c \Rightarrow k = 6$	6
17	a	$S[7]{=}a = a \Rightarrow k = 7$	**7**
18	b	$S[8]{=}\# \neq b;\ k = \pi[7]{=}1;\ S[2]{=}b = b \Rightarrow k = 2$	2
19	a	$S[3]{=}a = a \Rightarrow k = 3$	3

$$\boxed{\pi(S) \;=\; \langle\, 0,0,1,2,3,0,1,\ 0,1,2,3,4,5,4,5,6,7,\ 2,3 \,\rangle}.$$

Observación: las primeras $m = 7$ entradas coinciden exactamente con el LPS de P calculado en el Enunciado 2.

12. (b) Ocurrencias detectadas

La condición de ocurrencia es $\pi[q] = m = 7$ en la zona del texto ($q \geq m + 2 = 9$). El único tal valor es

$$q = 17, \qquad \pi[17] = 7.$$

Conversión a índices de T. El carácter $T[i]$ ocupa la posición $m+1+i$ de S. Si la ocurrencia de largo m *termina* en q, entonces $T[i]$ con $i = q-(m+1)$ es el último carácter de la ocurrencia en el texto; su inicio (1-indexado) es

$$i - m + 1 = q - 2m, \qquad \text{shift} = \text{inicio} - 1 = q - 2m - 1.$$

Para $q = 17$: inicio en $T = 17 - 14 = 3$, shift $= 2$:

$$\boxed{T[3..9] = \texttt{ababaca} = P, \qquad \text{shift } 2}.$$

Esto *coincide* con el resultado del Enunciado 2 (única ocurrencia en el shift 2). No hay más posiciones con $\pi[q] = 7$ (los $\pi[18] = 2$ y $\pi[19] = 3$ corresponden a los prefijos `ab` y `aba`, incompletos).

13. (c) Por qué el # y el costo

Rol del centinela. El # es un carácter que no aparece en P. Si se computara π sobre PT sin el #, un prefijo de P podría casarse con un *sufijo que cruza la frontera* entre P y T, produciendo $\pi[q]$ mayores a los reales y *falsas ocurrencias*. Por ejemplo, sin #, al procesar el inicio de T se podría heredar el k acumulado de P y reportar matches que no existen. El # corta esa conexión: como # no aparece en T, ningún sufijo de la zona del texto puede extender un prefijo más allá de la posición $m + 1$, lo que garantiza

$$\pi[q] \leq m \qquad \text{para todo } q \text{ en la zona del texto,}$$

y $\pi[q] = m$ indica exactamente una ocurrencia que termina en q.

Costo. Se computa π de un string de largo $n + m + 1$ con el mismo argumento amortizado de siempre (el contador k crece a lo más una unidad por paso y decrece solo vía π):

Tiempo $\Theta(n + m)$, Espacio adicional $\Theta(n + m)$

para guardar π. Es el mismo costo asintótico de KMP, con la ventaja de usar una única pasada sobre $S = P\#T$.